

Trabajo práctico: Sistema de Alquiler de Bicicletas

Alumno: Nicolás Maure

Docente: Ing. Juan Manuel Goya

1. Enunciado

“Una empresa de bicicletas compartidas gestiona estaciones distribuidas en la ciudad. Cada estación tiene un código, una dirección y una capacidad máxima de bicicletas. Cada bicicleta tiene un número de serie, un estado (disponible, en uso, en mantenimiento) y pertenece siempre a una estación en un momento dado. Los usuarios se registran con DNI, nombre y uno o más medios de pago (tarjetas). Cada viaje registra qué usuario retiró qué bicicleta, desde qué estación de origen, hacia qué estación de destino, con fecha/hora de inicio y fin — un viaje no existe sin la bicicleta ni sin el usuario que lo realizó. Cada estación puede requerir múltiples tareas de mantenimiento programado a lo largo del tiempo, cada una con fecha y descripción, realizada por un operario técnico (con legajo y nombre); un operario puede realizar muchas tareas, y una tarea de mantenimiento la realiza un solo operario.”

2. Etapa 1: Extracción de requerimientos

Analice buscando los **sustantivos** (candidatos a entidad), sus **características** (atributos) y los **verbos** que los vinculan (vínculos).

2.1 Entidades y atributos

Entidad	Tipo	Atributos	Clave
ESTACION	Fuerte	codigo_estacion, direccion, capacidad_max	codigo_estacion
BICICLETA	Fuerte	numero_serie, estado (disponible / en uso / en mantenimiento)	numero_serie
USUARIO	Fuerte	dni, nombre, medio_pago (multivaluado: uno o más)	dni
VIAJE	Fuerte	id_viaje, fecha_inicio, fecha_fin	id_viaje
TAREA_MANTENIMIENTO	Débil (depende de ESTACION)	numero_tarea, fecha, descripcion	Parcial: numero_tarea
OPERARIO	Fuerte	legajo, nombre	legajo

2.2 Vínculos encontrados

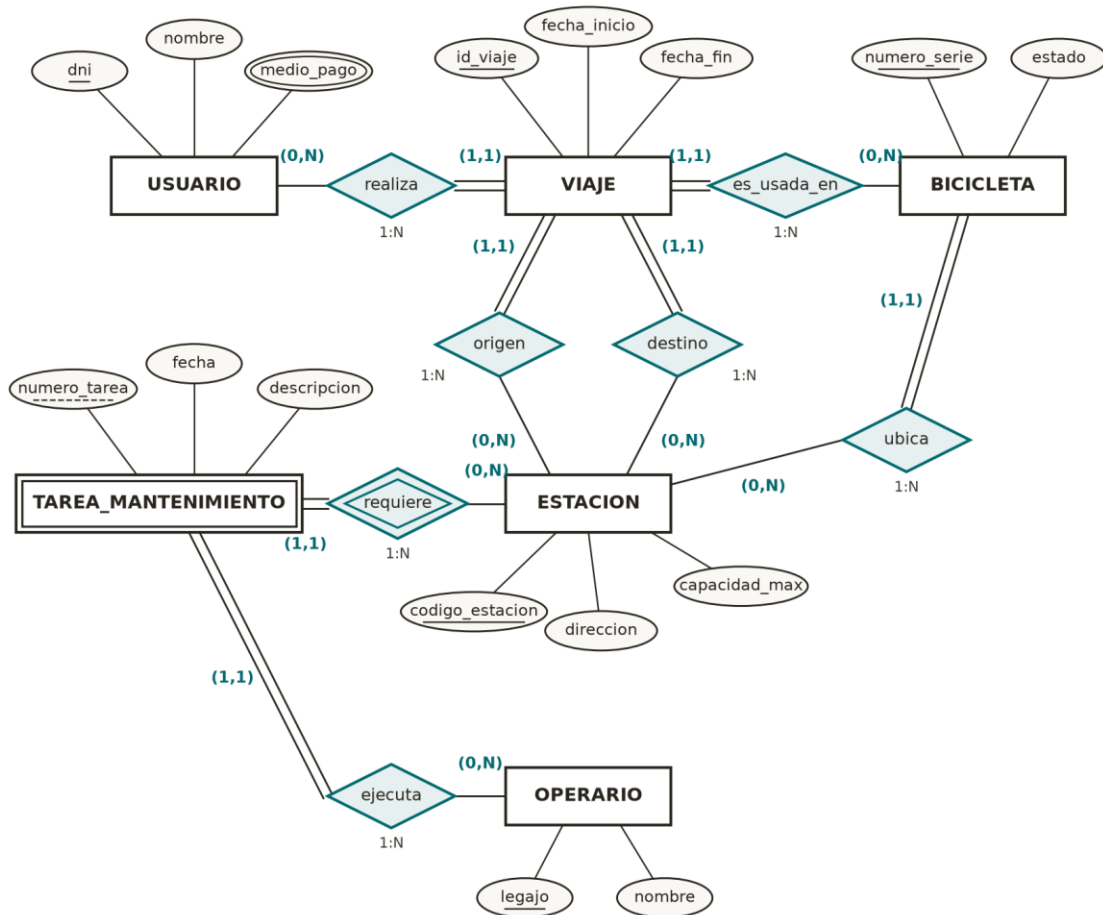
Vínculo	Entidades	Tipo	Frase del enunciado
ubica	ESTACION – BICICLETA	1:N	“pertenece siempre a una estación en un momento dado”
realiza	USUARIO – VIAJE	1:N	“qué usuario retiró...”
es_usada_en	BICICLETA – VIAJE	1:N	“...qué bicicleta”
origen	ESTACION – VIAJE	1:N	“desde qué estación de origen”
destino	ESTACION – VIAJE	1:N	“hacia qué estación de destino”
requiere	ESTACION – TAREA_MANTENIMIENTO	1:N (identificador)	“cada estación puede requerir múltiples tareas”
ejecuta	OPERARIO – TAREA_MANTENIMIENTO	1:N	“un operario puede realizar muchas tareas, y una tarea la realiza un solo operario”

2.3 Decisiones de diseño

- **medio_pago como atributo multivaluado.** No tiene atributos propios más allá del número de tarjeta, por lo que no se justifica una entidad. Se representa con óvalo doble y, en el modelo relacional, genera su propia tabla.
- **VIAJE como entidad y no como vínculo.** Conecta tres entidades a la vez (usuario, bicicleta y estación con dos roles) y tiene varios atributos propios; además, conceptualmente tiene identidad propia, por lo que se le asigna el identificador `id_viaje`.
- **Dos vínculos entre VIAJE y ESTACION.** La misma entidad participa con dos **roles** distintos: estación de origen y estación de destino. No alcanza con un solo vínculo ni con atributos sueltos “origen/destino”.
- **“Un viaje no existe sin la bicicleta ni sin el usuario”.** Se expresa como **participación total** de VIAJE en realiza y es_usada_en (cardinalidad (1,1) y línea doble). En el modelo físico se traduce en FK NOT NULL con ON DELETE RESTRICT. VIAJE no se modela como débil porque tiene clave propia (`id_viaje`).
- **TAREA_MANTENIMIENTO como entidad débil.** Las tareas se numeran dentro de cada estación (`numero_tarea` es clave parcial) y se identifican junto con el código de la estación propietaria mediante el vínculo identificador requiere.
- **Nombres de vínculos.** Para no repetir “realiza”, el vínculo operario–tarea se denomina ejecuta.

3. Etapa 2: Diagrama Entidad-Relación

Diagrama Entidad-Relación - Sistema de Alquiler de Bicicletas Compartidas



Referencias: rectángulo = entidad | rectángulo doble = entidad débil | rombo = vínculo | rombo doble = vínculo identificador
 óvalo = atributo | subrayado = clave | subrayado punteado = clave parcial | óvalo doble = multivaluado
 línea doble = participación total | (mín,máx) junto a cada entidad

Resumen de cardinalidades

ESTACION	(0,N)	—ubica—	(1,1)	BICICLETA
USUARIO	(0,N)	—realiza—	(1,1)	VIAJE
BICICLETA	(0,N)	—es_usada_en—	(1,1)	VIAJE
ESTACION	(0,N)	—origen—	(1,1)	VIAJE [rol: origen]
ESTACION	(0,N)	—destino—	(1,1)	VIAJE [rol: destino]
ESTACION	(0,N)	==requiere==	(1,1)	TAREA_MANTENIMIENTO [débil]
OPERARIO	(0,N)	—ejecuta—	(1,1)	TAREA_MANTENIMIENTO
USUARIO	(1,N)	medio_pago		[atributo multivaluado]

Lectura de ejemplo: una estación ubica de 0 a N bicicletas, y cada bicicleta está ubicada en exactamente una estación (1,1). Una estación puede ser origen de 0 a N viajes, y cada viaje tiene exactamente una estación de origen.

4. Etapa 3: Transformación al modelo relacional

```
ESTACION(codigo_estacion PK, direccion, capacidad_max)
OPERARIO(legajo PK, nombre)
USUARIO(dni PK, nombre)
BICICLETA(numeroSerie PK, estado, codigo_estacion FK→ESTACION NOT NULL)
USUARIO_MEDIO_PAGO(dni FK→USUARIO, numero_tarjeta, PK=(dni, numero_tarjeta))

VIAJE(id_viaje PK,
      fecha_inicio, fecha_fin,
      dni_usuario      FK→USUARIO      NOT NULL,
      numeroSerie_bici FK→BICICLETA     NOT NULL,
      codigo_est_origen FK→ESTACION     NOT NULL,
      codigo_est_destino FK→ESTACION    NOT NULL)

TAREA_MANTENIMIENTO(codigo_estacion FK→ESTACION, numero_tarea,
                    fecha, descripcion,
                    legajo_operario FK→OPERARIO NOT NULL,
                    PK=(codigo_estacion, numero_tarea))
```

Reglas de transformación aplicadas

- **Entidad fuerte** → **tabla** con sus atributos simples; el atributo clave pasa a ser la PK.
- **Vínculo 1:N** → **FK en la tabla del lado N**. Por eso codigo_estacion va en BICICLETA, y dni_usuario, numeroSerie_bici y las dos estaciones van en VIAJE.
- **Participación total (1,1)** → **FK NOT NULL**. La bicicleta siempre tiene estación; el viaje siempre tiene usuario, bicicleta, origen y destino.
- **Vínculo con roles** → **una FK por rol**. VIAJE tiene dos FK distintas hacia ESTACION: codigo_est_origen y codigo_est_destino.
- **Atributo multivaluado** → **tabla nueva** con la PK de la entidad dueña más el valor: USUARIO_MEDIO_PAGO(dni, numero_tarjeta).
- **Entidad débil** → **tabla con PK compuesta** por la PK de la entidad fuerte (codigo_estacion) y la clave parcial (numero_tarea).

5. Etapa 4: Script DDL

Las tablas se crean en orden: primero las que no tienen claves foráneas (ESTACION, OPERARIO, USUARIO) y luego las que las referencian. El script completo, con datos de prueba y consultas, se entrega en el archivo bicicletas_compartidas.sql.

```
DROP DATABASE IF EXISTS bicicletas_compartidas;
CREATE DATABASE bicicletas_compartidas
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE bicicletas_compartidas;

CREATE TABLE estacion (
  codigo_estacion VARCHAR(10) PRIMARY KEY,
  direccion        VARCHAR(150) NOT NULL,
  capacidad_max    INT          NOT NULL CHECK (capacidad_max > 0)
);

CREATE TABLE operario (
  legajo INT PRIMARY KEY,
```

```

    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE usuario (
    dni    VARCHAR(10)  PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

-- Vínculo 1:N "ubica": la FK va del lado N (bicicleta).
-- NOT NULL porque la bicicleta pertenece SIEMPRE a una estación.
CREATE TABLE bicicleta (
    numero_serie    VARCHAR(20) PRIMARY KEY,
    estado           VARCHAR(20) NOT NULL DEFAULT 'DISPONIBLE',
    codigo_estacion VARCHAR(10) NOT NULL,
    CONSTRAINT chk_bici_estado
        CHECK (estado IN ('DISPONIBLE', 'EN USO', 'EN MANTENIMIENTO')),
    CONSTRAINT fk_bici_estacion FOREIGN KEY (codigo_estacion)
        REFERENCES estacion (codigo_estacion) ON DELETE RESTRICT
);

-- Atributo multivaluado medio_pago -> tabla propia.
-- PK compuesta (dni, numero_tarjeta). CASCADE: la tarjeta no tiene
-- sentido sin su usuario.
CREATE TABLE usuario_medio_pago (
    dni            VARCHAR(10),
    numero_tarjeta VARCHAR(20),
    PRIMARY KEY (dni, numero_tarjeta),
    CONSTRAINT fk_medio_usuario FOREIGN KEY (dni)
        REFERENCES usuario (dni) ON DELETE CASCADE
);

-- VIAJE: entidad con identidad propia (id_viaje).
-- Dos FK a ESTACION con roles distintos: origen y destino.
-- RESTRICT en todas: no se pierde el historial de viajes.
-- El viaje se registra completo (inicio, fin, origen y destino), como
-- pide el enunciado; por eso fecha_fin y destino son NOT NULL.
CREATE TABLE viaje (
    id_viaje          INT PRIMARY KEY,
    fecha_inicio       DATETIME NOT NULL,
    fecha_fin          DATETIME NOT NULL,
    dni_usuario        VARCHAR(10) NOT NULL,
    numero_serie_bici  VARCHAR(20) NOT NULL,
    codigo_est_origen  VARCHAR(10) NOT NULL,
    codigo_est_destino VARCHAR(10) NOT NULL,
    CONSTRAINT chk_viaje_fechas
        CHECK (fecha_fin >= fecha_inicio),
    CONSTRAINT fk_viaje_usuario FOREIGN KEY (dni_usuario)
        REFERENCES usuario (dni) ON DELETE RESTRICT,
    CONSTRAINT fk_viaje_bici FOREIGN KEY (numero_serie_bici)
        REFERENCES bicicleta (numero_serie) ON DELETE RESTRICT,
    CONSTRAINT fk_viaje_origen FOREIGN KEY (codigo_est_origen)
        REFERENCES estacion (codigo_estacion) ON DELETE RESTRICT,
    CONSTRAINT fk_viaje_destino FOREIGN KEY (codigo_est_destino)
        REFERENCES estacion (codigo_estacion) ON DELETE RESTRICT
);

-- TAREA_MANTENIMIENTO: entidad débil de ESTACION.
-- PK = clave de la fuerte (codigo_estacion) + clave parcial (numero_tarea).
-- CASCADE hacia ESTACION (débil); RESTRICT hacia OPERARIO.
CREATE TABLE tarea_mantenimiento (
    codigo_estacion VARCHAR(10),
    numero_tarea     INT,
    fecha            DATE NOT NULL,
    descripcion       VARCHAR(200) NOT NULL,
    legajo_operario  INT NOT NULL,

```

```

PRIMARY KEY (codigo_estacion, numero_tarea),
CONSTRAINT fk_tarea_estacion FOREIGN KEY (codigo_estacion)
REFERENCES estacion (codigo_estacion) ON DELETE CASCADE,
CONSTRAINT fk_tarea_operario FOREIGN KEY (legajo_operario)
REFERENCES operario (legajo) ON DELETE RESTRICT
);

```

Políticas ON DELETE justificadas

Clave foránea	Política	Justificación
viaje → usuario	RESTRICT	No se puede borrar un usuario con viajes: se perdería el historial.
viaje → bicicleta	RESTRICT	El viaje no existe sin la bicicleta; se conserva el historial.
viaje → estación (origen y destino)	RESTRICT	Una estación con viajes registrados no debe desaparecer.
bicicleta → estación	RESTRICT	Toda bicicleta debe estar en una estación; primero hay que reubicarla.
tarea_mantenimiento → estación	CASCADE	Es entidad débil: sus tareas no tienen sentido sin la estación.
tarea_mantenimiento → operario	RESTRICT	No se borra un operario que tiene tareas registradas.
usuario_medio_pago → usuario	CASCADE	Las tarjetas pertenecen al usuario; se eliminan con él.

Tiene tres restricciones CHECK. `capacidad_max > 0` ya estaba en el script de clase; se agregaron dos: `estado` solo acepta los tres valores del enunciado, y `fecha_fin` no puede ser anterior a `fecha_inicio`. Además, `descripcion` es NOT NULL porque el enunciado indica que cada tarea tiene fecha y descripción.

6. Prueba en MariaDB

Se ejecutó en MariaDB (en XAMPP se carga desde la pestaña **Importar** de phpMyAdmin). Se crearon las siete tablas, se cargaron datos de prueba (4 estaciones, 4 bicicletas, 2 usuarios con 3 tarjetas, 3 viajes y 5 tareas; las bicicletas quedan ubicadas en la estación de destino de su último viaje) y se ejecutaron las siguientes pruebas de integridad:

Prueba	Resultado esperado	Resultado obtenido
Borrar un usuario con viajes	Rechazo por RESTRICT	ERROR 1451 (fk_viaje_usuario)
Insertar una bicicleta con estado 'ROTA'	Rechazo por CHECK	ERROR 4025 (chk_bici_estado)
Insertar una bicicleta en la estación E99 (inexistente)	Rechazo por FK	ERROR 1452 (fk_bici_estacion)
Borrar la estación E01 (tiene bicicletas y viajes)	Rechazo por RESTRICT	ERROR 1451 (fk_bici_estacion)
Insertar un viaje con fin anterior al inicio	Rechazo por CHECK	ERROR 4025 (chk_viaje_fechas)
Borrar la estación E04 (solo tiene tareas)	Se borra y sus tareas se eliminan en cascada	OK: 0 tareas restantes de E04

Consulta de verificación: listado de viajes con el nombre del usuario y la dirección de las estaciones de origen y destino. Requiere unir dos veces la tabla ESTACION con alias distintos, uno por cada rol.

```
SELECT v.id_viaje, u.nombre AS usuario, v.numero_serie_bici AS bici,
       eo.direccion AS origen, ed.direccion AS destino,
       v.fecha_inicio, v.fecha_fin
FROM viaje v
JOIN usuario u ON u.dni = v.dni_usuario
JOIN estacion eo ON eo.codigo_estacion = v.codigo_est_origen
JOIN estacion ed ON ed.codigo_estacion = v.codigo_est_destino
ORDER BY v.id_viaje;
```

id	usuario	bici	origen	destino	inicio	fin
1	Ana Pérez	B001	Av. 9 de Julio 1200	Plaza 25 de Mayo s/n	01/10 08:00	01/10 08:25
2	Luis Gómez	B001	Plaza 25 de Mayo s/n	Av. 9 de Julio 1200	02/10 18:10	02/10 18:40
3	Ana Pérez	B002	Av. 9 de Julio 1200	Costanera y Av. 3 de Abril	01/10 19:00	01/10 19:35

7. Observaciones

- **Capacidad máxima.** El esquema guarda capacidad_max, pero un CHECK no puede contar filas de otra tabla. Controlar que una estación no supere su capacidad requeriría un trigger o una validación en la aplicación.
- **Viaje registrado al finalizar.** El enunciado indica que cada viaje registra origen, destino, inicio y fin; por eso fecha_fin y codigo_est_destino son NOT NULL y el viaje se registra completo al devolver la bicicleta. Esto evita la contradicción de un viaje sin fin pero con destino ya cargado. En el script de clase fecha_fin admitía NULL; este cambio se hizo para que el modelo sea coherente.
- **Bicicleta “en uso”.** Como el enunciado dice que la bicicleta pertenece siempre a una estación, mientras está en uso se interpreta que codigo_estacion conserva la estación de la que fue retirada, hasta que se registra el viaje.
- **Medios de pago (1,N).** El modelo indica que cada usuario tiene al menos una tarjeta, pero una FK no puede obligar a que exista una fila en usuario_medio_pago. Esa regla quedaría a cargo de la aplicación o de un trigger.
- **Identificadores agregados.** id_viaje y numero_tarea no aparecen en el enunciado: son decisiones de diseño tomadas en clase para identificar cada viaje y cada tarea.
- **Datos de tarjetas.** Modelo simplificado para el ejercicio: en los datos de prueba los números están enmascarados; en un sistema real se guarda un token o solo los últimos 4 dígitos, nunca el número completo.

Browser address bar: nicomaure.canadacentral.cloudapp.azure.com/phpmyadmin/index.php?route=/database/designer&d...

PHPMyAdmin interface showing the database structure for **bicicletas_compartidas**.

Database Structure:

- bicicletas_compartidas tarea_mantenimiento**
 - codigo_estacion : varchar(10)
 - numero_tarea : int(11)
 - fecha : date
 - descripcion : varchar(200)
 - legajo_operario : int(11)
- bicicletas_compartidas bicicleta**
 - numeroSerie : varchar(20)
 - estado : varchar(20)
 - codigo_estacion : varchar(10)
- bicicletas_compartidas operario**
 - legajo : int(11)
 - nombre : varchar(100)
- bicicletas_compartidas viaje**
 - id_viaje : int(11)
 - fecha_inicio : datetime
 - fecha_fin : datetime
 - dni_usuario : varchar(10)
 - numeroSerieBici : varchar(20)
 - codigo_est_origen : varchar(10)
 - codigo_est_destino : varchar(10)
- bicicletas_compartidas usuario**
 - dni : varchar(10)
 - nombre : varchar(100)
- bicicletas_compartidas usuario_medio_pago**
 - dni : varchar(10)
 - numero_tarjeta : varchar(20)
- bicicletas_compartidas estacion**
 - codigo_estacion : varchar(10)
 - direccion : varchar(150)
 - capacidad_max : int(11)

Relationships:

- bicicletas_compartidas tarea_mantenimiento** is linked to **bicicletas_compartidas bicicleta** (green line).
- bicicletas_compartidas tarea_mantenimiento** is linked to **bicicletas_compartidas operario** (blue line).
- bicicletas_compartidas viaje** is linked to **bicicletas_compartidas usuario** (yellow line).
- bicicletas_compartidas viaje** is linked to **bicicletas_compartidas usuario_medio_pago** (yellow line).
- bicicletas_compartidas viaje** is linked to **bicicletas_compartidas estacion** (green line).
- bicicletas_compartidas viaje** is linked to **bicicletas_compartidas bicicleta** (purple line).